

To define a hash function for an object, define the `__hash__` method.

```
class HashToOne(object):
    def __hash__(self):
        return 1
HTO = HashToOne()
print(hash(HTO))
```

Output:
1

To set an object as not hashable, set `__hash__` to `None`.

```
class NotHashable(object):
    __hash__ = None
NH = NotHashable()
print(hash(NH))
```

Output:

```
Traceback (most recent call last):
  File "dict_sample.py", line 94, in <module>
    print(hash(NH))
TypeError: unhashable type: 'NotHashable'
```

Define your own hash with caution, as it could easily mislead the actual implementation of datatypes and one may encounter subtle issues. Read the blog at <https://www.asmeurer.com/blog/posts/what-happens-when-you-mess-with-hashing-in-python/> to understand the challenges and their solutions.

▪ Hash tables

Hash tables are the engines behind dicts and sets making it quicker to achieve $O(1)$. Here only the keys must be hashable. Each position in the hash table is called a bucket.

Figure 1 gives one way to handle a hash collision but there are several other ways, which we will discuss later.

Once the hash value is generated, it's passed through a

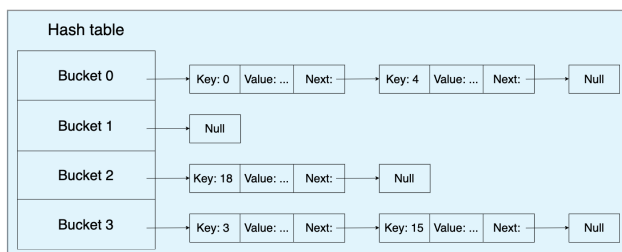


Figure 1: Hash table (Credit: <https://tentousandmeters.com/blog/python-behind-the-scenes-10-how-python-dictionaries-work/>)

mathematical process to get the bucket number placed. Here's the process.

- Simple version: `bucket_index = hash(key) % number_of_buckets`
- Yet another version: `bucket_index = hash(key) % prime_number`

Hash table is initially built as a sparse array that usually comes with half or one-third empty cells.

- Sample dict: `d = {'timmy': 'red', 'barry': 'green', 'guido': 'blue'}`
- Code structure:

```
struct dict {
    long num_items;

    dict_entry* items; /* pointer to array */
}

struct dict_entry {
    long hash;

    PyObject* key;

    PyObject* value;
}
Stored as:
items = [['-', '-', '-'],
         [-8522787127447073495, 'barry', 'green'],
         ['-', '-', '-'],
         ['-', '-', '-'],
         ['-', '-', '-'],
         [-9092791511155847987, 'timmy', 'red'],
         ['-', '-', '-'],
         [-6480567542315338377, 'guido', 'blue']]
```

After Python 3.6 the structure was changed to make it compact and memory efficient.

- Code structure:

```
struct dict {
    long num_items;

    // new PyPy dictionary is split in two arrays
    variable_int *sparse_array;
    dict_entry* compact_array;
}

struct dict_entry {
```